College of Engineering Trivandrum
Department of Electrical and Electronics Engineering


# Design and Optimisation of a Closed-Loop DC-DC Boost Converter

Using Genetic Algorithm-Tuned PI Control

**Project Report**

Author:

**Varun Sathya**
BTech EEE

April 25, 2026

# Contents


**1 Introduction** **2**

**2 Boost Converter Parameters** **2**

**3 Open Loop Boost Converter** **2**

3.1 Circuit Description ..... 2

3.2 Theoretical Output Voltage ..... 3

3.3 Simulated Result ..... 3

**4 Closed Loop PI Controller** **3**

4.1 Motivation ..... 4

4.2 Control Loop Structure ..... 4

4.3 Plant Transfer Function ..... 5

4.3.1 Linearisation Problem ..... 5

4.3.2 Transfer Function Used for GA Tuning ..... 5

**5 Genetic Algorithm Parameter Optimisation** **6**

5.1 Motivation ..... 6

5.2 Algorithm Overview ..... 6

5.3 Search Space ..... 6

5.4 Cost Function ..... 7

5.5 MATLAB Implementation ..... 7

5.6 Simulink Integration ..... 9

**6 Simulation Configuration** **10**

**7 Results and Discussion** **10**

7.1 Open Loop ..... 10

7.2 Closed Loop with GA-Optimised PI ..... 10

7.3 Comparison ..... 11

**8 Conclusion** **11**

# 1 Introduction

A DC-DC boost converter steps up a lower DC input voltage to a higher DC output voltage. In an ideal lossless converter, the output voltage is determined entirely by the input voltage and the switch duty cycle. In practice however, switching losses, diode forward voltage drops, and parasitic resistances cause the actual output to fall below the theoretical value. A fixed duty cycle cannot correct for these losses, and open loop control is therefore insufficient for precision voltage regulation.

This project addresses that limitation through three stages. First, an open loop boost converter is simulated to demonstrate the gap between theoretical and actual output voltage. Second, a closed loop PI controller is introduced to regulate the output to the 20V target by automatically adjusting the duty cycle. Third, the PI controller gains are optimised using a Genetic Algorithm (GA), a metaheuristic search method that minimises a cost function encoding the desired output behaviour. The result is a systematic, automated method for tuning a power converter controller without manual trial and error.

# 2 Boost Converter Parameters

The circuit was designed to step up a 12V input to a 20V output at 100W with a switching frequency of 25 kHz. The full set of design parameters is given in Table 1.

Table 1: Boost Converter Design Parameters

| **Parameter** | **Symbol** | **Value** |
|---|---|---|
| Input Voltage | $V_{in}$ | 12 V |
| Output Voltage (Target) | $V_{out}$ | 20 V |
| Switching Frequency | $f_s$ | 25 kHz |
| Output Power | $P$ | 100 W |
| Duty Cycle (Nominal) | $D$ | 0.4 |
| Output Current | $I_o$ | 5 A |
| Inductor Current | $I_L$ | 8.33 A |
| Inductor | $L$ | 76.8 $\mu$H |
| Capacitor | $C$ | 400 $\mu$F |
| Load Resistance | $R$ | 4 $\Omega$ |
| Ripple in Output Voltage | $\Delta V_o$ | 0.2 V (1%) |
| Ripple in Inductor Current | $\Delta I_L$ | 2.5 A (30%) |

The nominal duty cycle is derived from the ideal boost converter voltage gain:

$$D = 1 - \frac{V_{in}}{V_{out}} = 1 - \frac{12}{20} = 0.4 \tag{1}$$

# 3 Open Loop Boost Converter

## 3.1 Circuit Description

The open loop converter was implemented in MATLAB Simulink using the Simscape Electrical library. The circuit consists of a 12V DC source, an inductor, an N-channel MOSFET, a diode, an output filter capacitor, and a 4Ω resistive load. The MOSFET gate is driven by a fixed-frequency PWM signal with duty cycle $D = 0.4$ and switching frequency $f_s = 25$ kHz. No feedback path is present.

## 3.2 Theoretical Output Voltage

For an ideal lossless boost converter operating in continuous conduction mode, the DC voltage gain is:

$$M = \frac{V_{out}}{V_{in}} = \frac{1}{1 - D} \tag{2}$$

Substituting $D = 0.4$ and $V_{in} = 12$ V:

$$V_{out} = \frac{12}{1 - 0.4} = \frac{12}{0.6} = 20 \text{ V} \tag{3}$$

## 3.3 Simulated Result

The simulated open loop output voltage settled at approximately **18.75 V**, which is 6.25% below the 20V target. This deficit arises from:

- Diode forward voltage drop reducing the effective output
- MOSFET on-state resistance causing conduction losses
- Inductor series resistance dissipating energy

These are inherent non-idealities that a fixed duty cycle cannot compensate for. This result confirms that closed loop feedback control is necessary to achieve the desired output voltage, and reflects what would be observed in a real physical circuit.

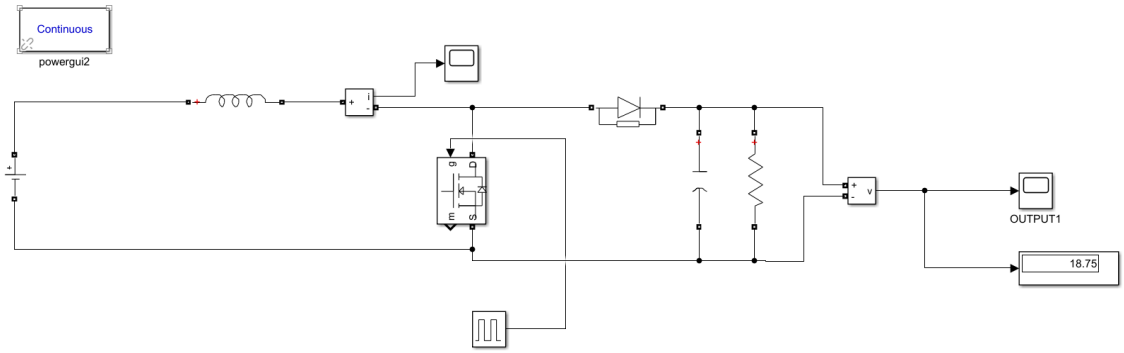


Figure 1: Open loop boost converter with an input voltage of 12V and a duty cycle of 0.4

s

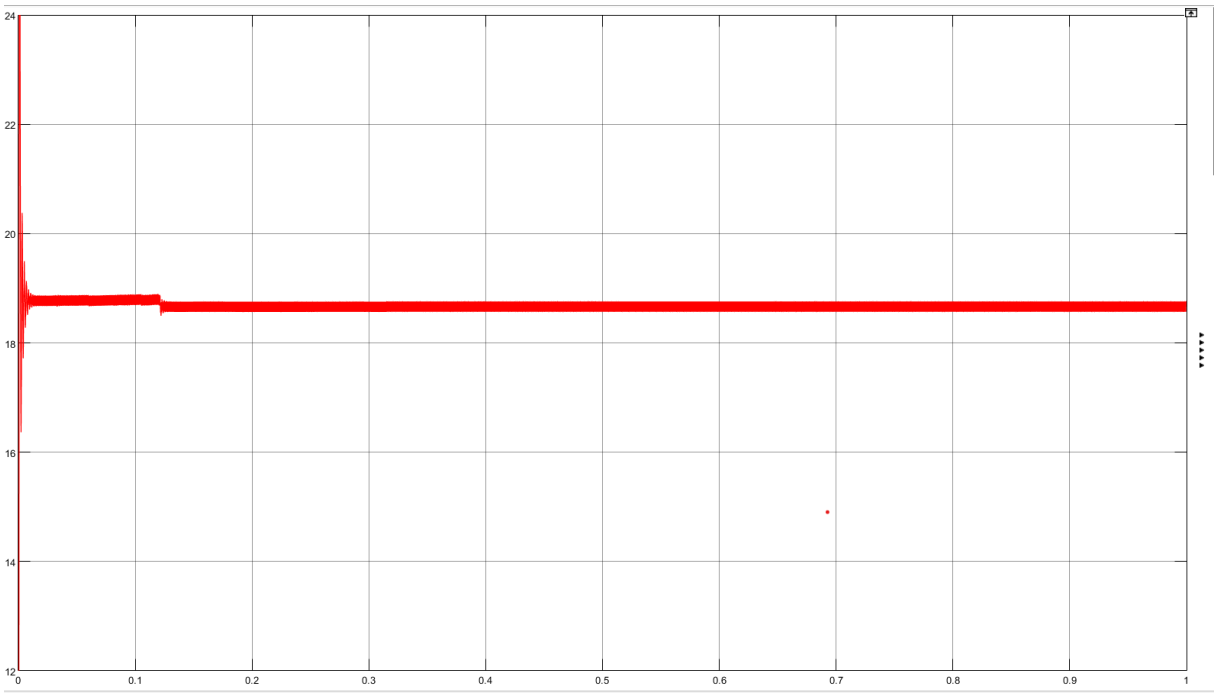


Figure 2: Graph of output voltage (Vo) of the open loop boost converter

# 4 Closed Loop PI Controller

## 4.1 Motivation

Since the open loop duty cycle of 0.4 yields only 18.75V, a higher duty cycle is needed to reach 20V. The exact duty cycle that achieves 20V under real circuit conditions is not known analytically because it depends on all the non-ideal loss mechanisms simultaneously. A PI controller solves this by continuously measuring the output voltage, computing the error from the 20V reference, and adjusting the duty cycle in real time until the error is driven to zero.

## 4.2 Control Loop Structure

The closed loop signal flow is as follows:

1. The output voltage $V_{out}$ is measured using a voltage measurement block
2. The error is computed: $e(t) = V_{ref} - V_{out}(t)$, where $V_{ref} = 20$ V
3. The error is normalised by a gain of $1/V_{ref} = 1/20$
4. The PI controller processes the normalised error and outputs a duty cycle command
5. The duty cycle command is clamped to $[0, 1]$ and fed to a PWM comparator
6. The comparator drives the MOSFET gate signal

The PI control law in the s-domain is:

$$C(s) = K_p + \frac{K_i}{s} \tag{4}$$

## 4.3 Plant Transfer Function

### 4.3.1 Linearisation Problem

MATLAB Simulink's built-in PID Tuner app attempts to automatically linearize the plant by computing the Jacobian of the model around an operating point. This fails for switched-mode power converter circuits because the MOSFET and diode are nonlinear switching elements. The error encountered was:

*Plant cannot be linearized. Use the Plant menu to create or select a new plant.*

To overcome this, the plant transfer function was derived analytically and provided manually to the tuner.

### 4.3.2 Transfer Function Used for GA Tuning

For the GA optimisation, the following form of the transfer function was used, derived from the converter voltage gain and circuit relationships:

$$G_{vd}(s) = \frac{V_{in}}{D'^2} \cdot \frac{-Ls + RD'^2}{LCR \cdot s^2 + L \cdot s + RD'^2} \tag{5}$$

In MATLAB:

```
Vin = 12; Vout_ref = 20;
L = 76.8e-6; C = 400e-6; R = 4;
D_prime = Vin / Vout_ref;  % = 0.6

num = (Vin / (D_prime^2)) * [-L, R*(D_prime^2)];
den = [(L*C*R), L, R*(D_prime^2)];
G_vd = tf(num, den);
```

Listing 1: Plant Transfer Function

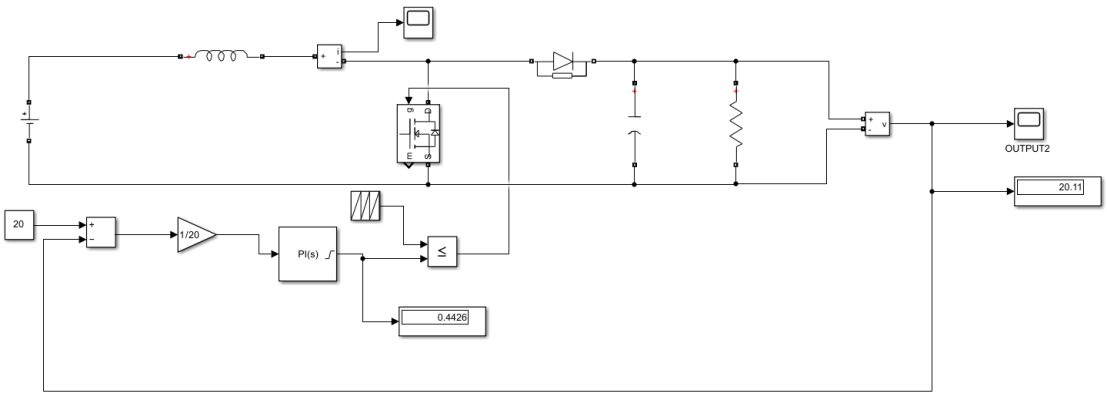


Figure 3: Closed loop boost converter using a PI controller with the tuned parameters: Kp = 0.001 and Ki = 9.9077

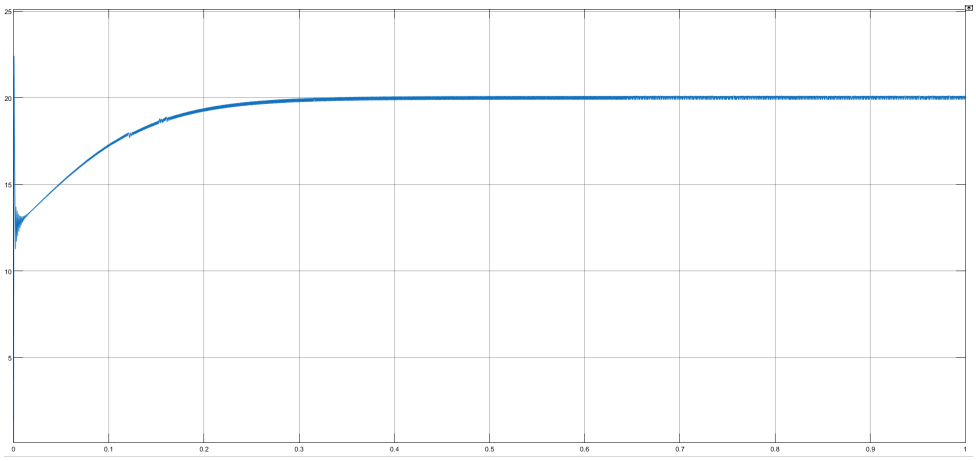


Figure 4: Graph of output voltage (Vo) for Closed loop boost coverter

# 5 Genetic Algorithm Parameter Optimisation

## 5.1 Motivation

Classical tuning methods such as Ziegler-Nichols produce a deterministic solution based on a linearized plant model. They do not directly optimize for time-domain criteria such as settling time or overshoot. They also break down in nonlinear or poorly modeled plants, where the linearized model no longer reflects the real system. A Genetic Algorithm is a population-based search method that mimics natural selection, and iteratively evolves candidate solutions by selecting, crossing, and mutating them to minimize a cost function. Because it searches the parameter space directly rather than relying on a plant model, it can optimize PID gains against any user-defined time-domain objective. This makes it well suited to cases where classical methods struggle.

## 5.2 Algorithm Overview

The Genetic Algorithm starts with a randomly generated population of candidate solutions (in this case, pairs of $[K_p, K_i]$ values). Each candidate is evaluated by the cost function. The fittest candidates are selected, crossed over to produce offspring, and randomly mutated. This repeats over multiple generations until the population converges to a near-optimal solution.

## 5.3 Search Space

The GA was constrained to search within bounds chosen around the expected range of valid gains:

Table 2: GA Search Bounds

| Parameter | Lower Bound | Upper Bound |
|---|---|---|
| $K_p$ | 0.005 | 5 |
| $K_i$ | 5 | 50 |

## 5.4 Cost Function

The cost function encodes three objectives:

1. **Steady state accuracy**: heavily penalise solutions where the final output deviates from 20V
2. **Overshoot**: penalise solutions where the output exceeds 20V during transient
3. **ITAE**: among solutions that satisfy the above, prefer the one with fastest error convergence

The ITAE (Integral of Time-weighted Absolute Error) criterion is defined as:

$$\text{ITAE} = \int_0^\infty t \cdot |e(t)|\, dt \approx \sum_{k=0}^{N} t_k \cdot |e(t_k)| \cdot \Delta t \tag{6}$$

The full cost function is:

$$J(K_p, K_i) = 100 \cdot |V_{ref} - y(t_{end})| + 10 \cdot \max\big(0,\, \max(y) - V_{ref}\big) + \text{ITAE} \tag{7}$$

The large coefficients on the first two terms act as hard constraints. Any solution with significant steady state error or overshoot receives a score orders of magnitude worse than a well-behaved solution. Once GA eliminates those candidates, ITAE differentiates among the remaining good solutions.

## 5.5 MATLAB Implementation

```
function [Kp, Ki] = ga_pid_tuner()

clear; clc;
rng(42);

%% 1. Parameters
Vin = 12;
Vout_ref = 20;
L = 76.8e-6;
C = 400e-6;
R = 4;
beta = 1;
D_prime = Vin / Vout_ref;

%% 2. Plant Transfer Function
num = (Vin / (D_prime^2)) * [-L, R*(D_prime^2)];
den = [(L*C*R), L, R*(D_prime^2)];
G_vd = tf(num, den);
Plant = G_vd * beta;

%% 3. GA Setup
num_vars = 2;
lb = [0.005, 5];
ub = [0.5, 50];
```

```
25
26  options = optimoptions('ga', ...
27      'PopulationSize', 30, ...
28      'MaxGenerations', 100, ...
29      'MaxStallGenerations', 20, ...
30      'FunctionTolerance', 1e-4, ...
31      'Display', 'iter', ...
32      'UseParallel', false);
33
34  cost_func = @(K) evaluate_boost_pi(K, Plant, Vout_ref);
35  [optimal_K, ~] = ga(cost_func, num_vars, [], [], [], [], lb, ub,
        [], options);
36
37  Kp_final = optimal_K(1);
38  Ki_final = optimal_K(2);
39  fprintf('\nGA Results: Kp = %.6f, Ki = %.6f\n', Kp_final,
        Ki_final);
40
41  %% 4. Verification Plot
42  sys_cl = feedback(pid(Kp_final, Ki_final) * Plant, 1);
43  figure('Name', 'GA PID Tuning Result', 'NumberTitle', 'off');
        step(Vout_ref * sys_cl, 0.05); grid on;
44  yline(Vout_ref, 'r--', '20V Reference');
45  title(['GA Optimized Response | Kp: ', num2str(Kp_final), ', Ki:
        ', num2str(Ki_final)]);
46  ylabel('Output Voltage (V)');
47
48  %% Cost Function
49  function cost = evaluate_boost_pi(K, Plant, Vref)
50      Kp = K(1); Ki = K(2);
51      C_ctrl = pid(Kp, Ki);
52      sys_cl = feedback(C_ctrl * Plant, 1);
53
54      % Stability check
55      if ~isstable(sys_cl)
56          cost = 1e8;
57          return;
58      end
59
60      % Step response scaled to Vref
61      t = 0:1e-4:0.01;
62      y = Vref * step(sys_cl, t);
63
64      % Voltage based penalties
65      steady_state_error = abs(Vref - y(end));
66      overshoot = max(0, max(y) - Vref);
67
68      % ITAE
69      error = abs(Vref - y);
70      itae = sum(t .* error') * (t(2) - t(1));
71
```

```
    cost = 100*steady_state_error + 10*overshoot + itae;
end

  Kp = Kp_final;
  Ki = Ki_final;
end
```

Listing 2: Genetic Algorithm PI Tuner

## 5.6 Simulink Integration

The GA tuner was called from a MATLAB Function block in Simulink. A persistent variable ensures the GA runs only once at the first simulation timestep, after which the stored gains are passed to the PI controller block for every subsequent timestep without recomputation.

```
function [Kp, Ki] = pid_tuner()
    coder.extrinsic('pid_tuner_ga');

    persistent Kp_stored Ki_stored
    Kp = 0; Ki = 0;

    if isempty(Kp_stored)
        [Kp_stored, Ki_stored] = pid_tuner_ga();
    end

    Kp = Kp_stored;
    Ki = Ki_stored;
end
```

Listing 3: MATLAB Function Block for Simulink Integration

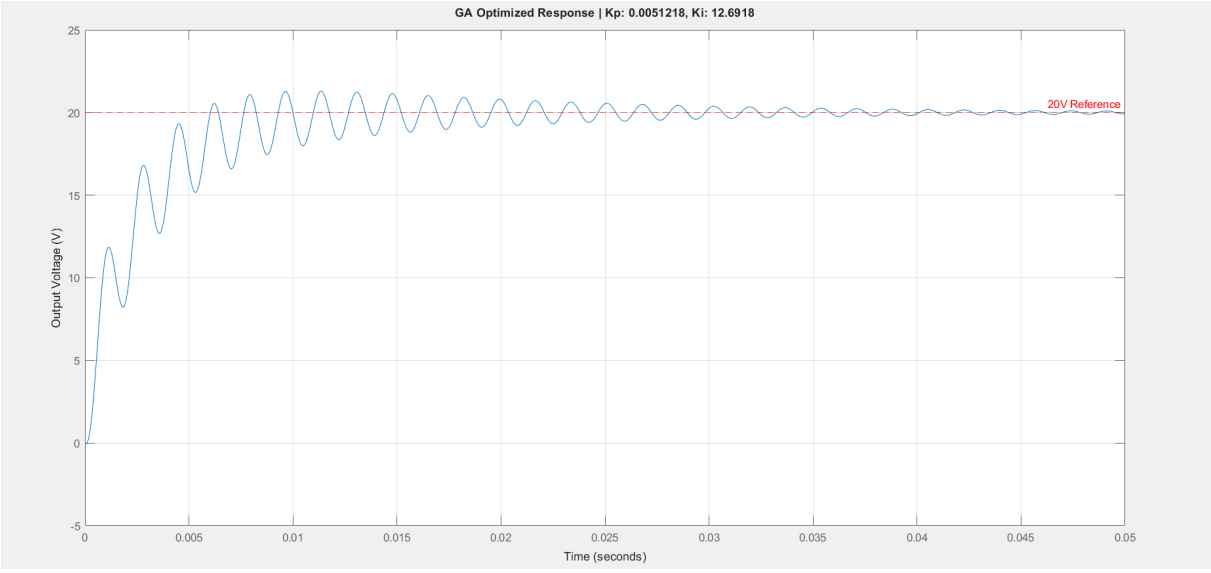


Figure 5: GA PID Tuning Result

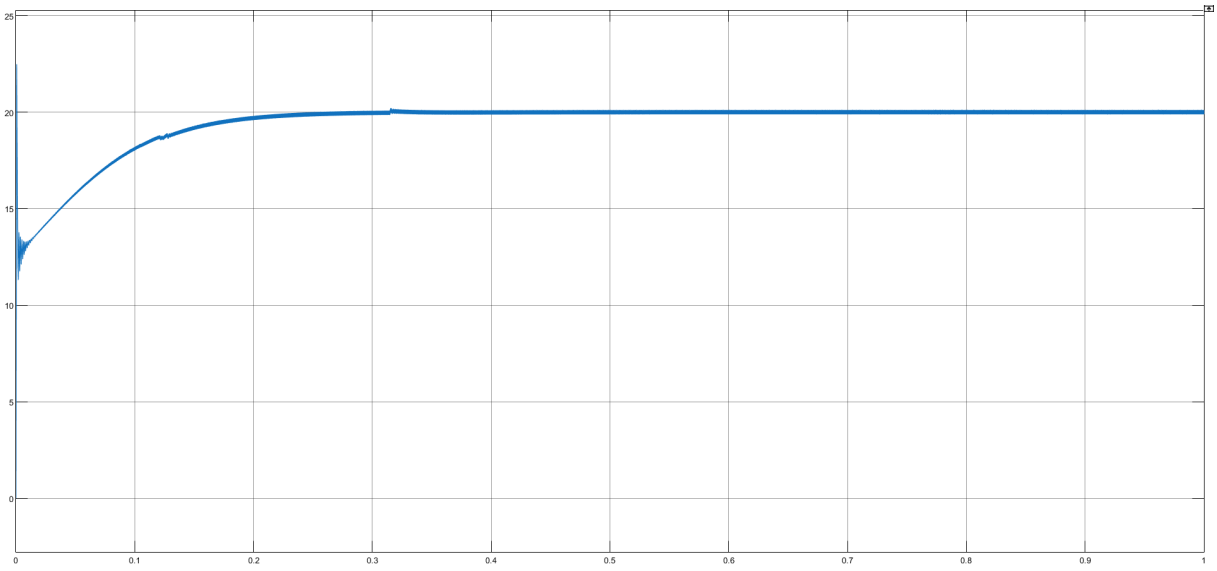


Figure 6: Output voltage (Vo) of boost coverter with genetic algorithm-based PID tuning

# 6 Simulation Configuration

Table 3: Simulink Simulation Settings

| **Setting** | **Value** |
|---|---|
| Solver | Variable step, continuous (auto) |
| PID sample time | $40 \times 10^{-6}$ s ($= 1/f_s$) |
| Anti-windup method | Clamping |
| Output saturation | $[0.1,\ 0.9]$ |

The PID sample time was set to match the switching period $T_s = 1/f_s = 40\,\mu$s so the controller updates exactly once per switching cycle. Anti-windup clamping was enabled with output limits of $[0, 1]$ to prevent the integrator from accumulating beyond the physical duty cycle range.

# 7 Results and Discussion

## 7.1 Open Loop

With a fixed duty cycle of $D = 0.4$, the simulated output voltage settled at **18.75 V**. The theoretical value from the ideal boost converter formula is 20V, giving a deficit of 1.25V (6.25%). This confirms the presence of real circuit losses that open loop control cannot compensate for.

## 7.2 Closed Loop with GA-Optimised PI

With the GA-tuned PI controller active, the output voltage converged to approximately **20 V** with an increased duty cycle of approximately **0.4376**.

Key results:

- Steady state output voltage reached the 20V reference
- The duty cycle adjustment of +0.0376 (from 0.4 to 0.4376) compensates for circuit non-idealities automatically
- The output settled within approximately 0.2s from startup
- Output ripple remained within acceptable bounds at the switching frequency

### 7.3 Comparison

Table 4: Performance Comparison Across Configurations

| **Metric** | **Open Loop** | **Closed Loop (GA PI)** |
|---|---|---|
| Output Voltage | 18.75 V | $\approx$ 20 V |
| Steady State Error | 1.25 V (6.25%) | $\approx$ 0 V |
| Duty Cycle | 0.4 (fixed) | 0.4376 (automatic) |
| Voltage Regulation | None | Active |
| Tuning Method | N/A | Genetic Algorithm + ITAE |

## 8 Conclusion

This project demonstrated the full design pipeline for a closed loop DC-DC boost converter, from open loop characterisation through to GA-optimised PI control. The open loop converter confirmed that circuit losses prevent the theoretical 20V output from being achieved with a fixed duty cycle. The closed loop PI controller resolved this by adapting the duty cycle in real time. The use of a Genetic Algorithm to tune the PI gains provided a systematic, automated approach that directly optimised for time-domain output voltage performance through the ITAE criterion, with hard penalties enforcing steady state accuracy and stability. The result was a well-regulated 20V output with minimal overshoot and fast settling, demonstrating the effectiveness of metaheuristic optimisation for power electronics controller design.